

Outline

- Overview

Domain specific languages

- Syntax trees
- Control flow diagrams
- Regular expressions

Domain Specific Language (DSL)



- Specialized in a particular **application domain**
- In contrast to general purpose languages (GPL), e.g. Java, which are broadly applicable across domains
- There is a wide variety of DSLs for common domains, such as HTML for web pages
- The line between **GPLs** and **DSLs** is not always sharp
 - A language may have specialized features for a particular domain but be applicable more broadly
 - Conversely, it may in principle be capable of broad application but in practice used primarily for a specific domain

Advantages and disadvantages of DSLs



- + Efficiency
- + Integration between developers and application domain experts
- + Independent of the target platform
- + Increased quality
- Narrows the use
- Learning curve
- Additional effort for configuration
- DSL creators need domain knowledge as well as language-development knowledge

Examples



- Web: **HTML**
- Document writing: LaTeX, Typst, **Markdown**
- Data exchange: **JSON, XML, YAML**
- Modeling: UML (PlantUML)
- Data management: SQL and its variants
- Business rules: Drools
- Hardware: Verilog, VHDL
- Build tools: Maven, **Gradle**
- Shell: sh, Bash, CSH, MS-DOS, Windows Terminal, PowerShell
- Numerical computation and simulation: MATLAB (commercial), GNU Octave, Scilab
- Various types of parsers and generators: Lex, YACC, GNU Bison, ANTLR

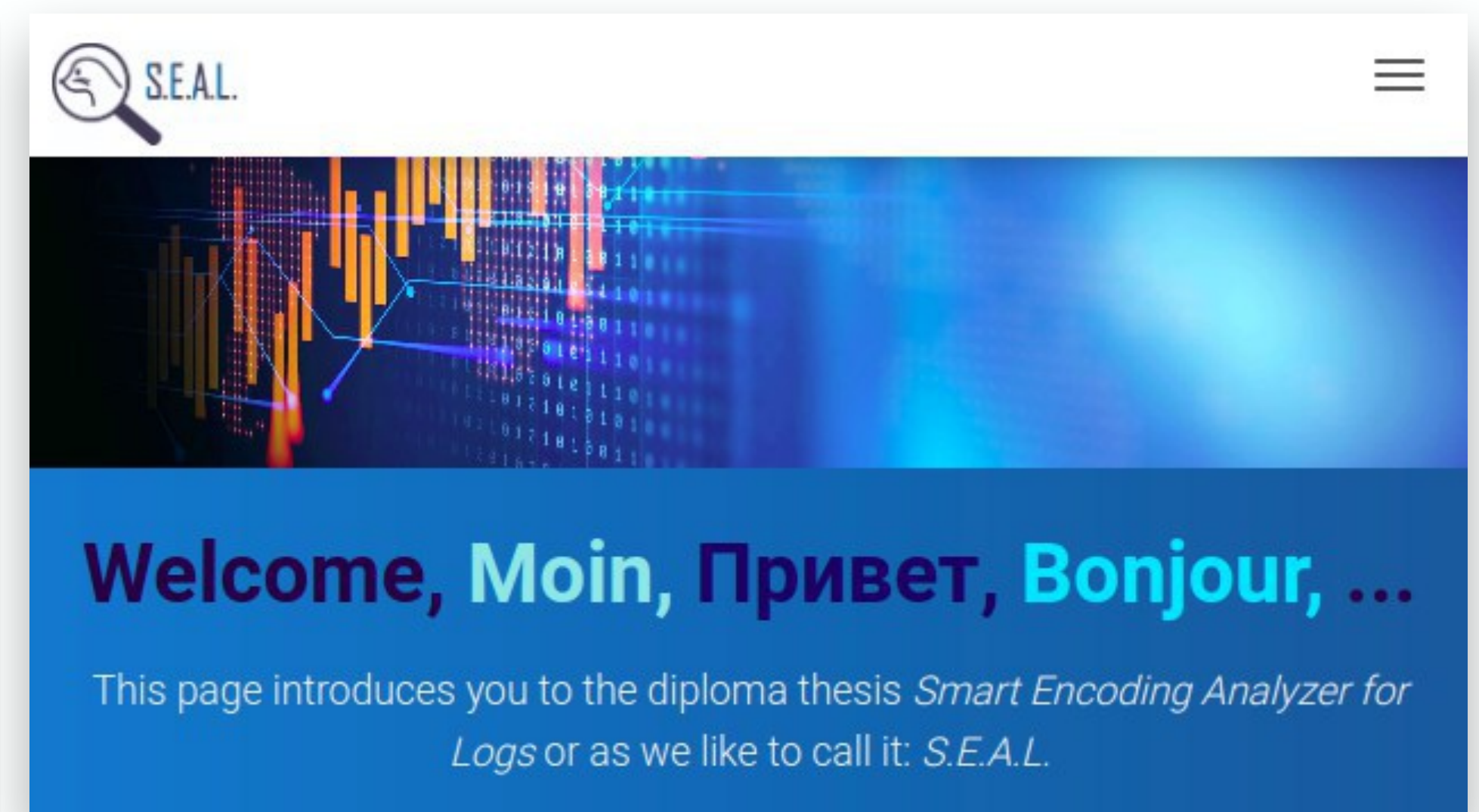
HTML: HyperText Markup Language

- HTML is the standard markup language for documents, designed to be displayed in a web browser
- It can be assisted by technologies such as **C**ascading **S**tyle **S**heets (CSS) and scripting languages such as JavaScript (JS)
- Cheatsheet: <https://htmlcheatsheet.com>
- Try it out: <https://html-online.com/editor>

Example

```
index.html
<!doctype html>
<html lang="en">

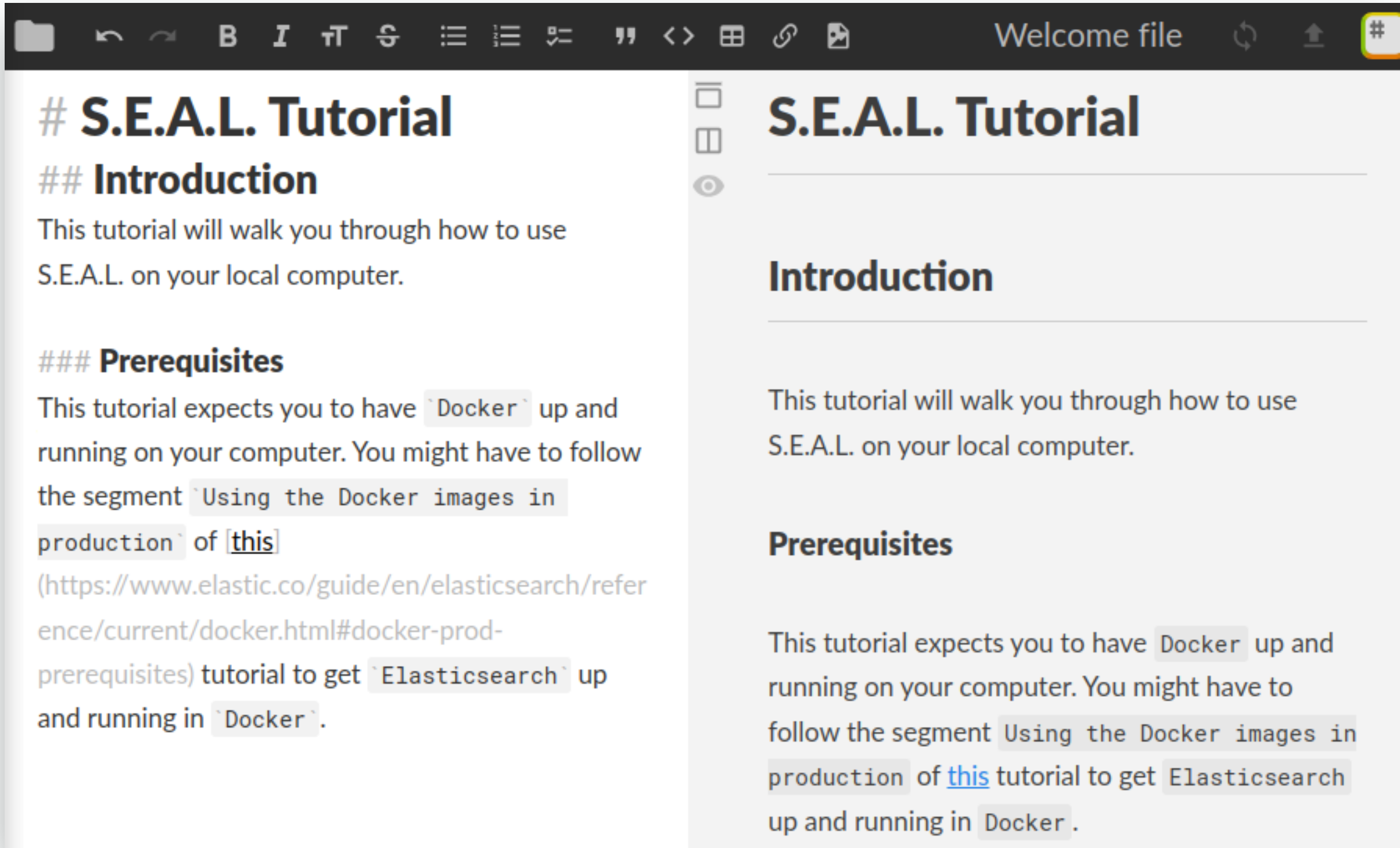
<head>
  ...
</head>
<body id="top" class="main_Background">
  ...
  <h2 class="color_title">Welcome, Moin, Привет, Bonjour, ...</h2>
  ...
</body>
</html>
```



Markdown

- Used primarily for documentation
- Try it out: <https://stackedit.io/app#>

Example



Element	Markdown Syntax
Heading	# H1 ## H2 ### H3
Bold	**bold text**
Italic	<i>*italicized text*</i>
Blockquote	> blockquote
Ordered List	1. First item 2. Second item 3. Third item
Unordered List	- First item - Second item - Third item
Code	`code`
Horizontal Rule	---
Link	[title](https://www.example.com)
Image	![alt text](image.jpg)

JSON: JavaScript Object Notation

- Open standard format
- Uses human readable text to transmit objects consisting of key value pairs
- Language independent data format
- Structure of JSON: <https://www.json.org/json-en.html>
- Try it out: <https://jsoneditoronline.org/>

Example

Class

Movie
– imdbID: String – title: String – simplePlot: String – genres: [String] – year: int – rating: String

Object

<u>hiddenFigures: Movie</u>
<pre>imdbID = "tt4846340" title = "Hidden Figures" simplePlot = "Three ..." genres = ["History", "Drama"] year = 2016 rating = "7.8"</pre>

JSON

<pre>{ "imdbID": "tt4846340", "title": "Hidden Figures", "simplePlot": "Three ...", "genres": ["History", "Drama"], "year": 2016, "rating": "7.8" }</pre>

- The extensible markup language defines a set of rules for encoding documents in a format that is both human-readable and machine-readable
- Try it out: https://www.tutorialspoint.com/online_xml_editor.htm

Example

Class

Movie
– imdbID: String – title: String – simplePlot: String – genres: [String] – year: int – rating: String

Object

<u>hiddenFigures: Movie</u>
imdbID = "tt4846340" title = "Hidden Figures" simplePlot = "Three ..." genres = ["History", "Drama"] year = 2016 rating = "7.8"

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<movie>
  <imdbID>tt4846340</imdbID>
  <title>Hidden Figures</title>
  <simplePlot>Three ...</simplePlot>
  <genres>
    <element>History</element>
    <element>Drama</element>
  </genres>
  <year>2016</year>
  <rating>7.8</rating>
</movie>
```


YAML: YAML Ain't Markup Language

- YAML is a human- and machine-readable data serialization format
- Often used for configurations and data exchange between languages with different data structures.
- Try it out: <https://codebeautify.org/yaml-editor-online>

Example

Class

Movie
– imdbID: String – title: String – simplePlot: String – genres: [String] – year: int – rating: String

Object

<u>hiddenFigures: Movie</u>
imdbID = "tt4846340" title = "Hidden Figures" simplePlot = "Three ..." genres = ["History", "Drama"] year = 2016 rating = "7.8"

YAML

```

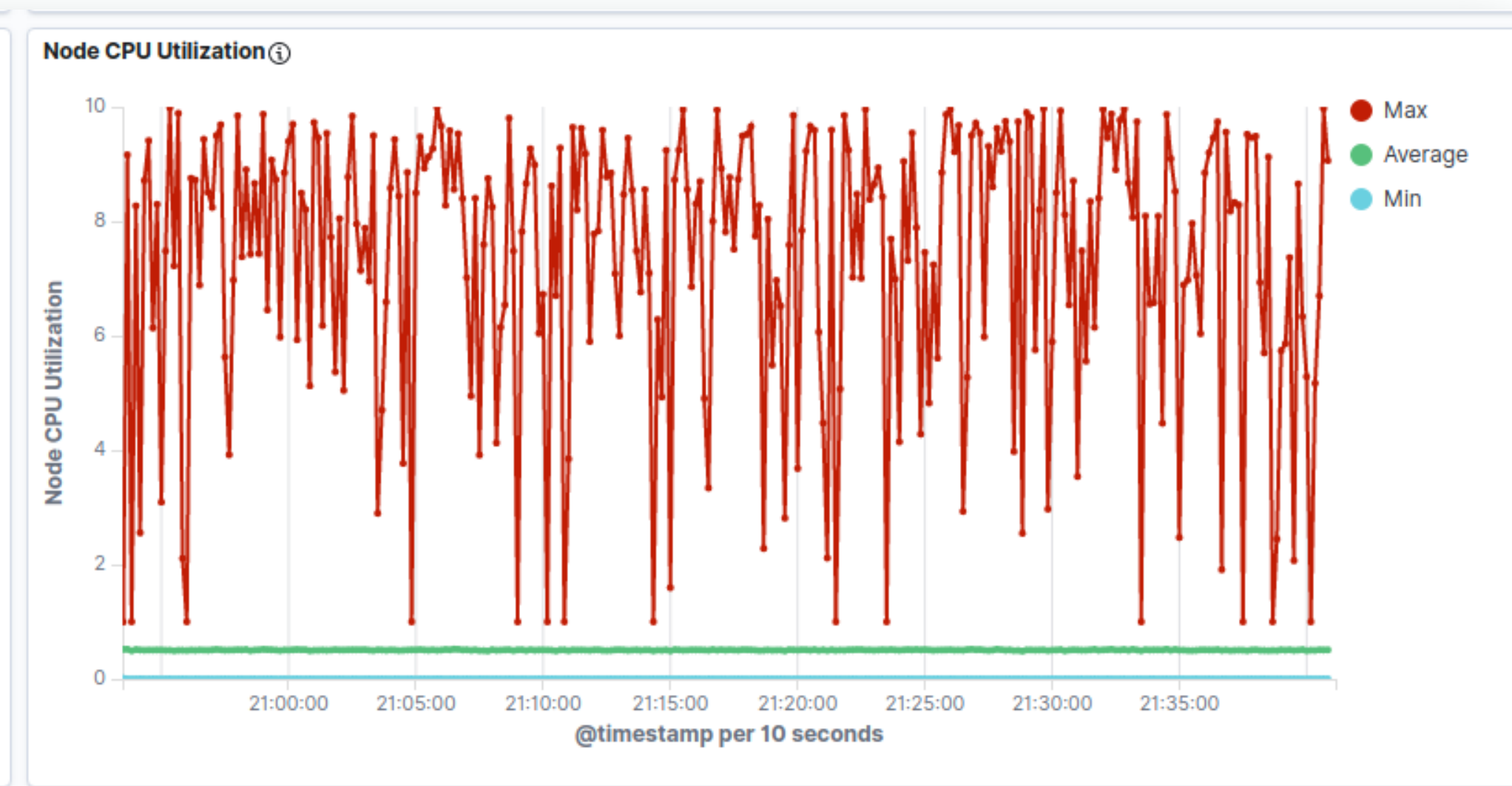
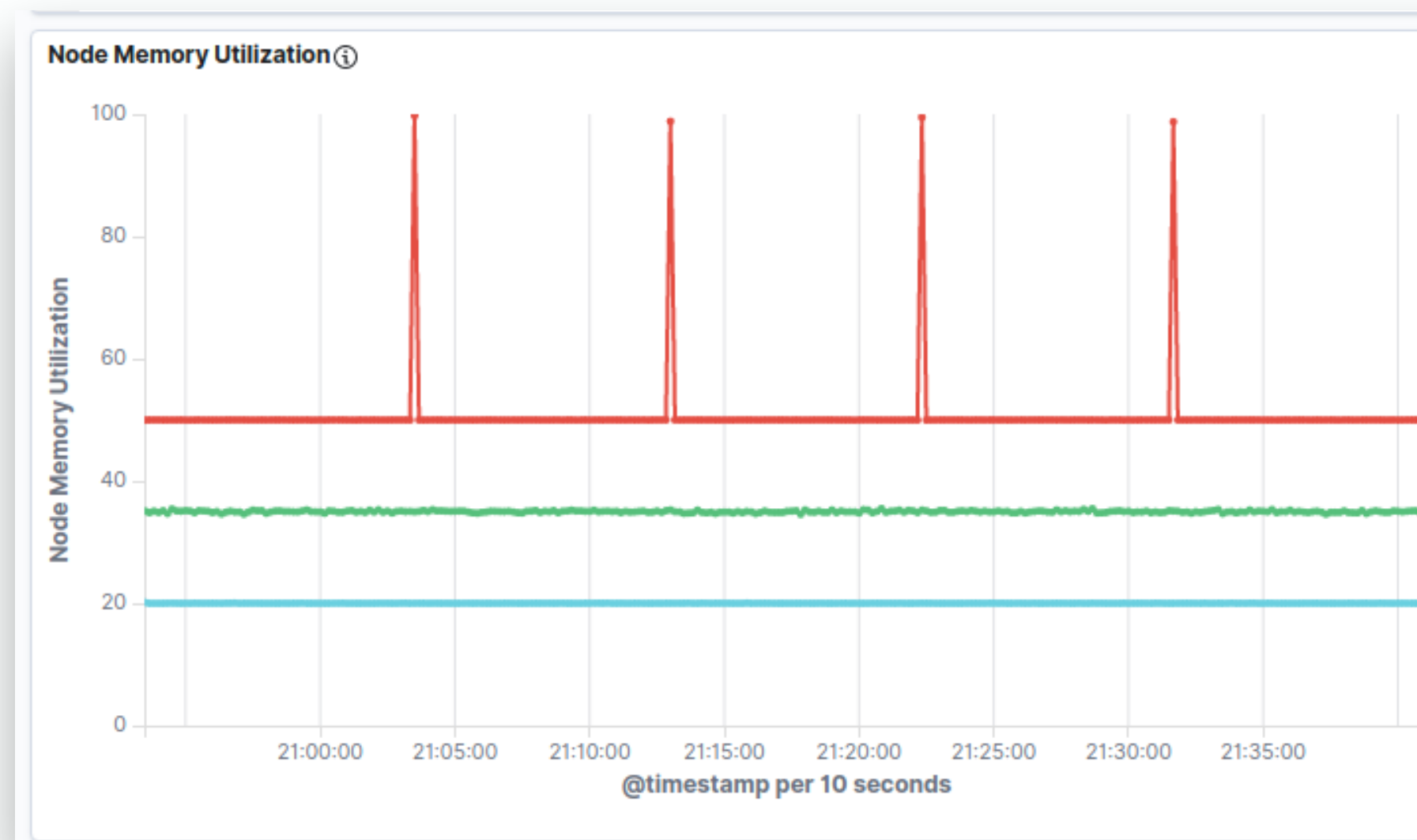
movie:
  imdb: "tt4846340"
  title: "Hidden Figures"
  simplePlot: "Three ..."
  genres:
    - "History"
    - "Drama"
  year: 2016
  rating: "7.8"

```

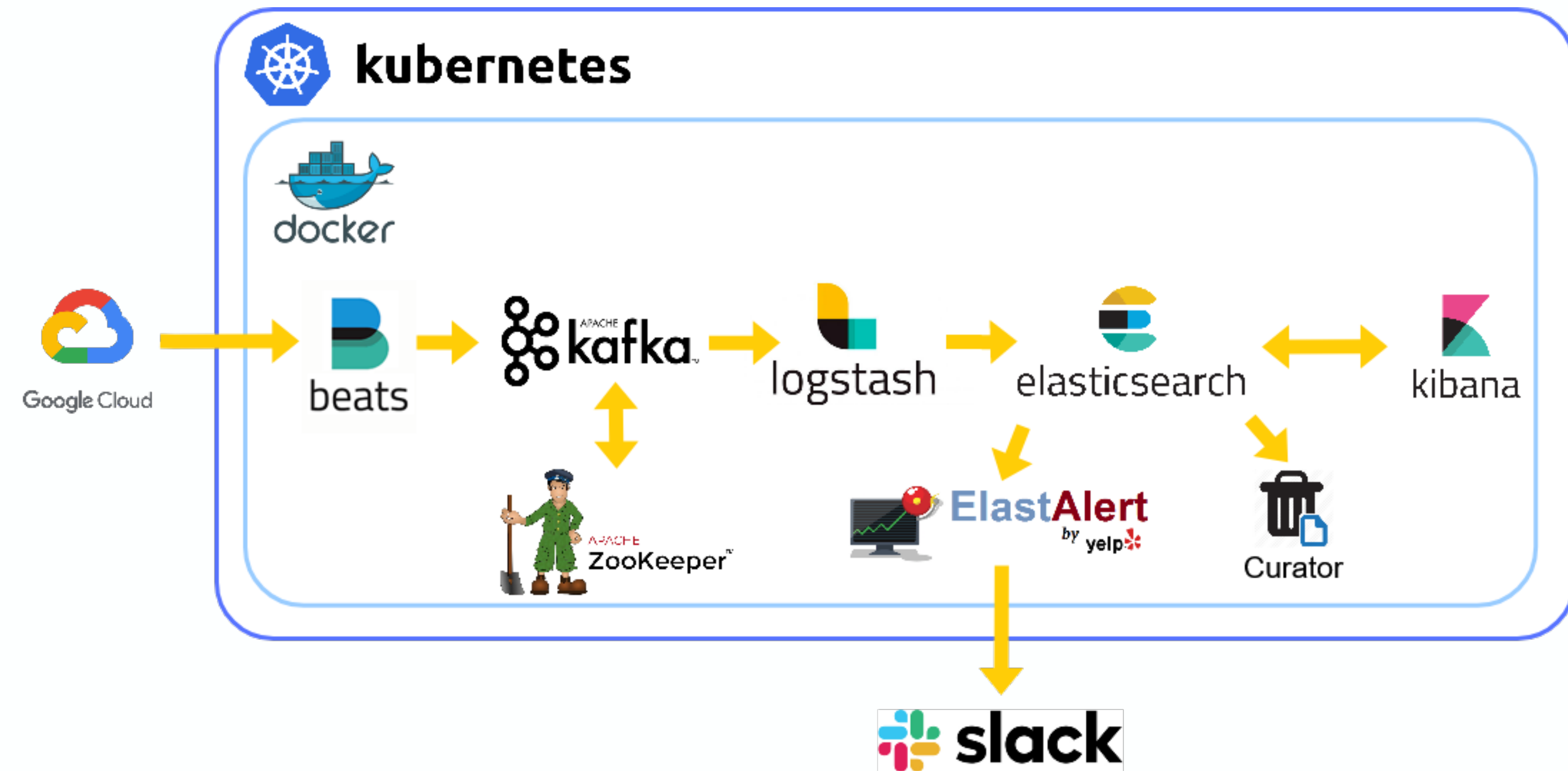
Example: Smart Encoding Analyzer for Logs (S.E.A.L.)

input.json

```
{
  "correlation_id": "01f97d55-517e-48c3-86c4-c8ae0fd9f2ca",
  "function": "status",
  "host": "ip-172-31-16-5",
  "message": "Currently 0 Services are finished.",
  "pid": 546,
  "service": "Session",
  "tid": 768,
  "time": "2020-07-05T14:16:00.490692803+00:00",
  "type": "INFO",
  "unit": "DistributedMultiStreamTranscodeService"
}
```



Example: Dataflow of S.E.A.L.



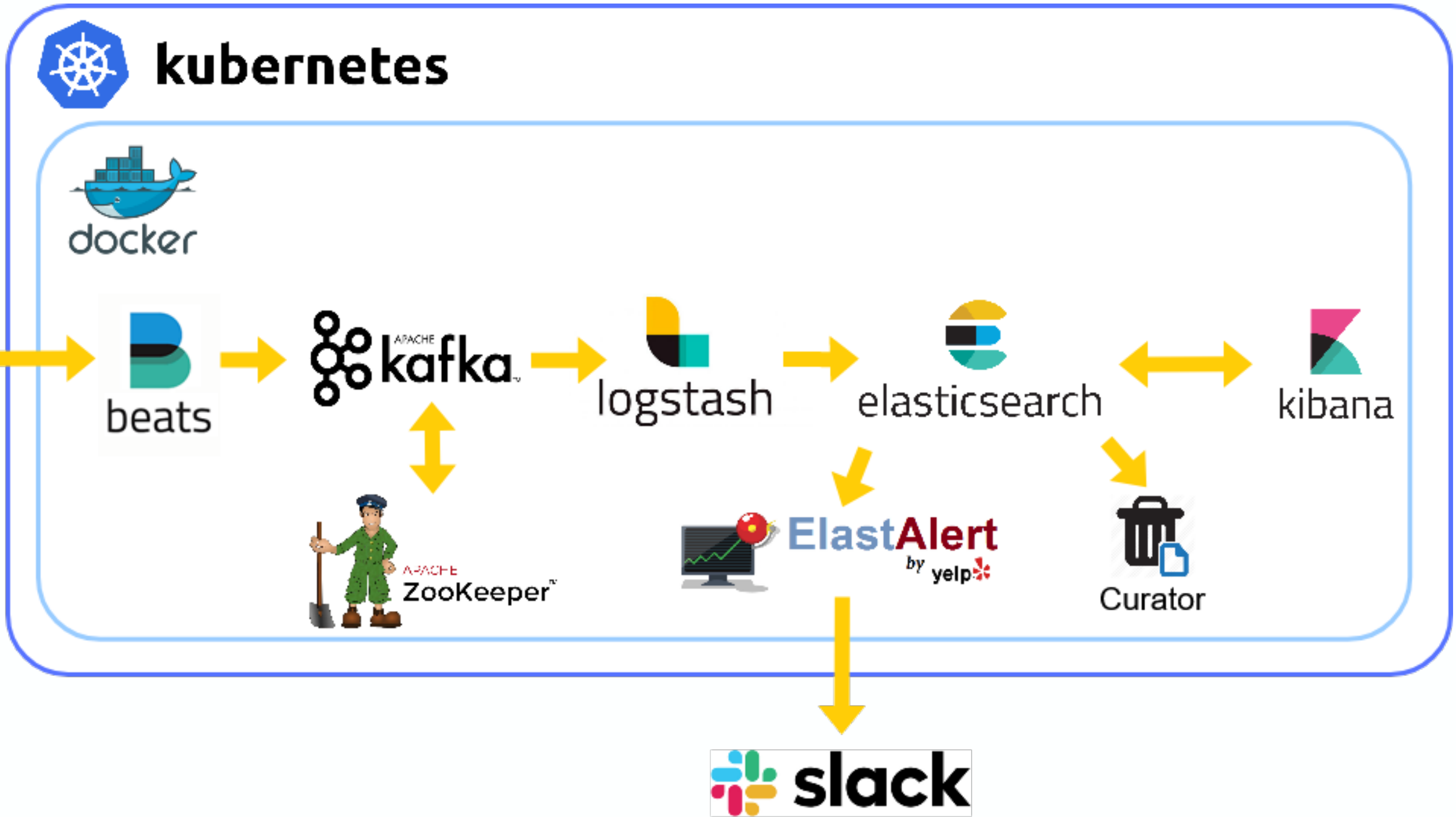
Example: Dataflow of S.E.A.L.

Input: 100.000 log files in the JSON format



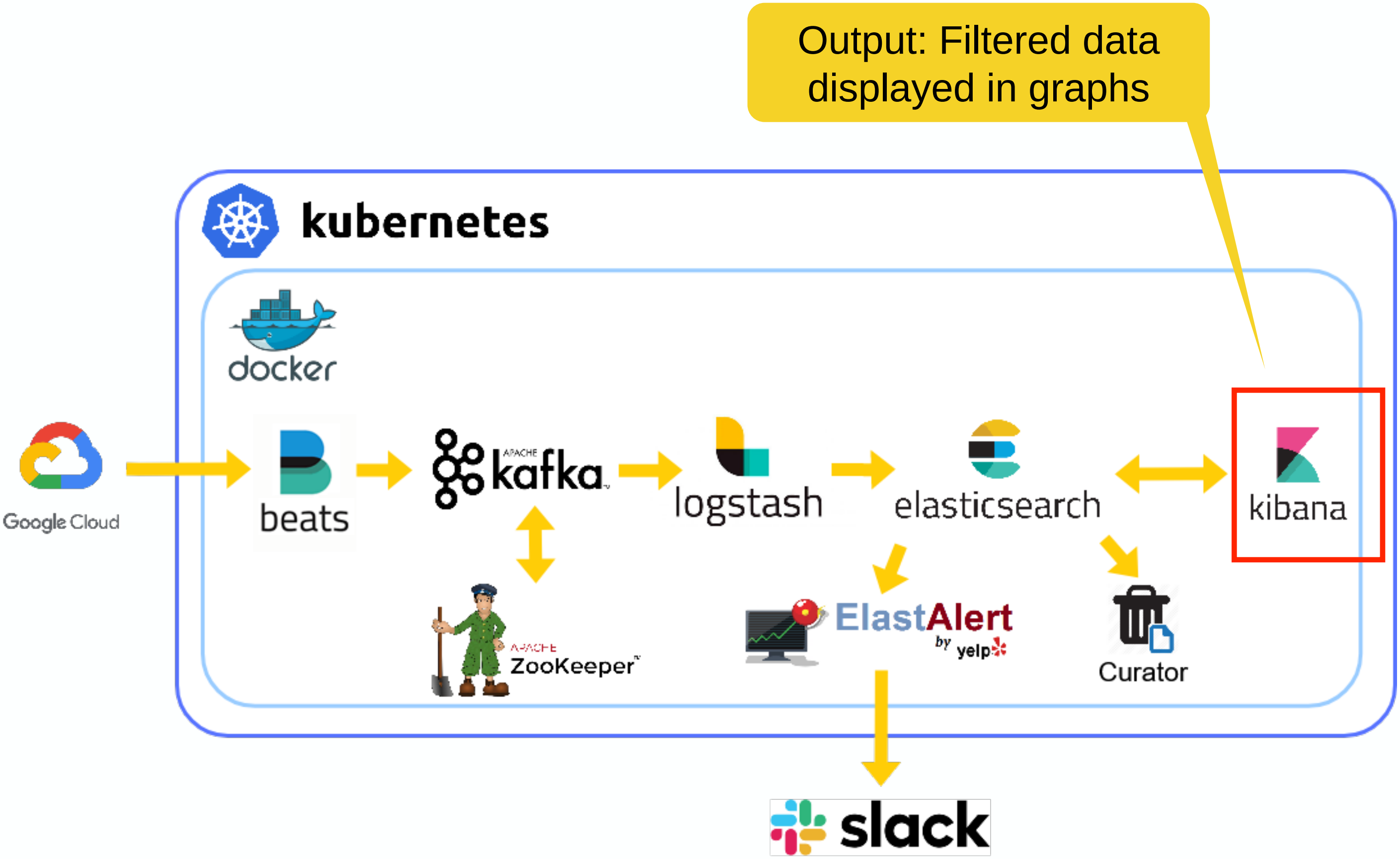
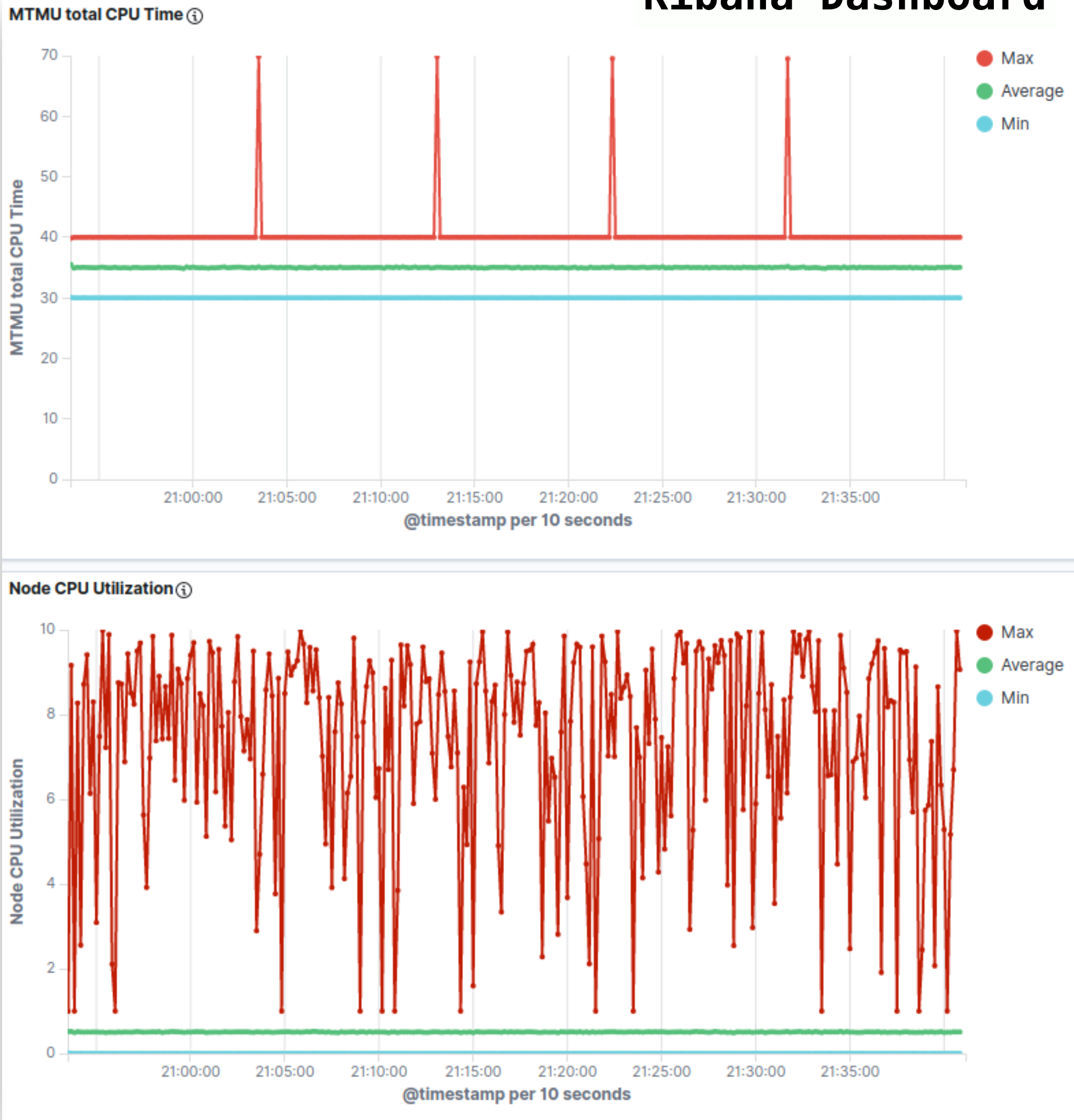
input.json

```
{
  "correlation_id": "01f97d55-517e-48c3-86c4-c8ae0fd9f2ca",
  "function": "status",
  "host": "ip-172-31-16-5",
  "message": "Currently 0 Services are finished.",
  "pid": 546,
  "service": "Session",
  "tid": 768,
  "time": "2020-07-05T14:16:00.490692803+00:00",
  "type": "INFO",
  "unit": "DistributedMultiStreamTranscodeService"
}
```



Example: Dataflow of S.E.A.L.

Kibana-Dashboard

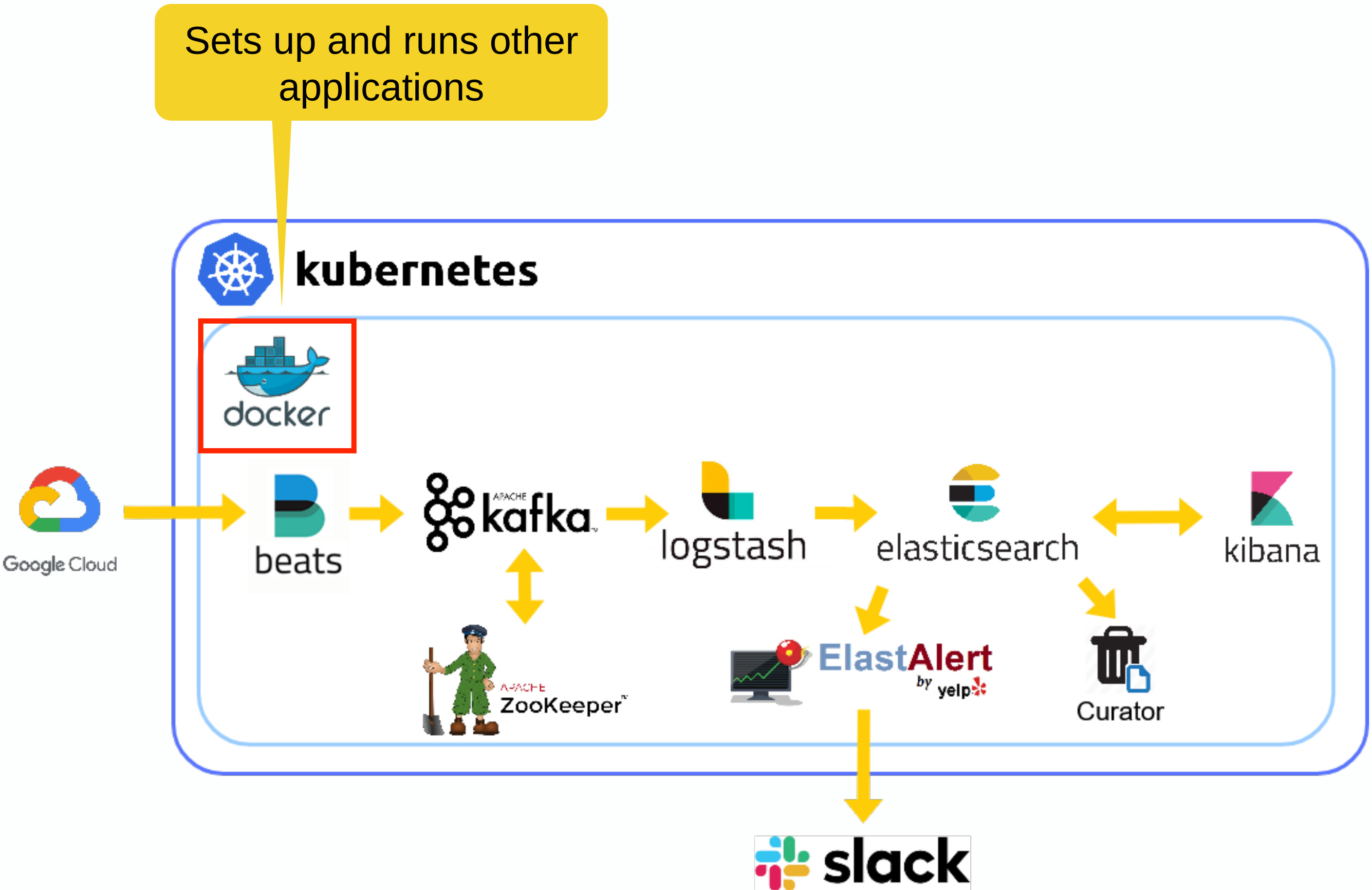


Example: Dataflow of S.E.A.L.

`docker-compose.yml`

```
version: '3.3'
services:
  ...

  kibana:
    image: docker.elastic.co/kibana/kibana:${ELK_VERSION}
    container_name: kibana
    volumes:
      - type: bind
        source: ./kibana/kibana.yml
        target: /usr/share/kibana/kibana.yml
        read_only: true
    networks:
      sealnet:
        ipv4_address: 172.28.1.4
    ports:
      - "5601:5601"
    depends_on:
      - elasticsearch
    networks:
      sealnet:
        ipam:
          driver: default
          config:
            - subnet: 172.28.0.0/16
    volumes:
      ...
```

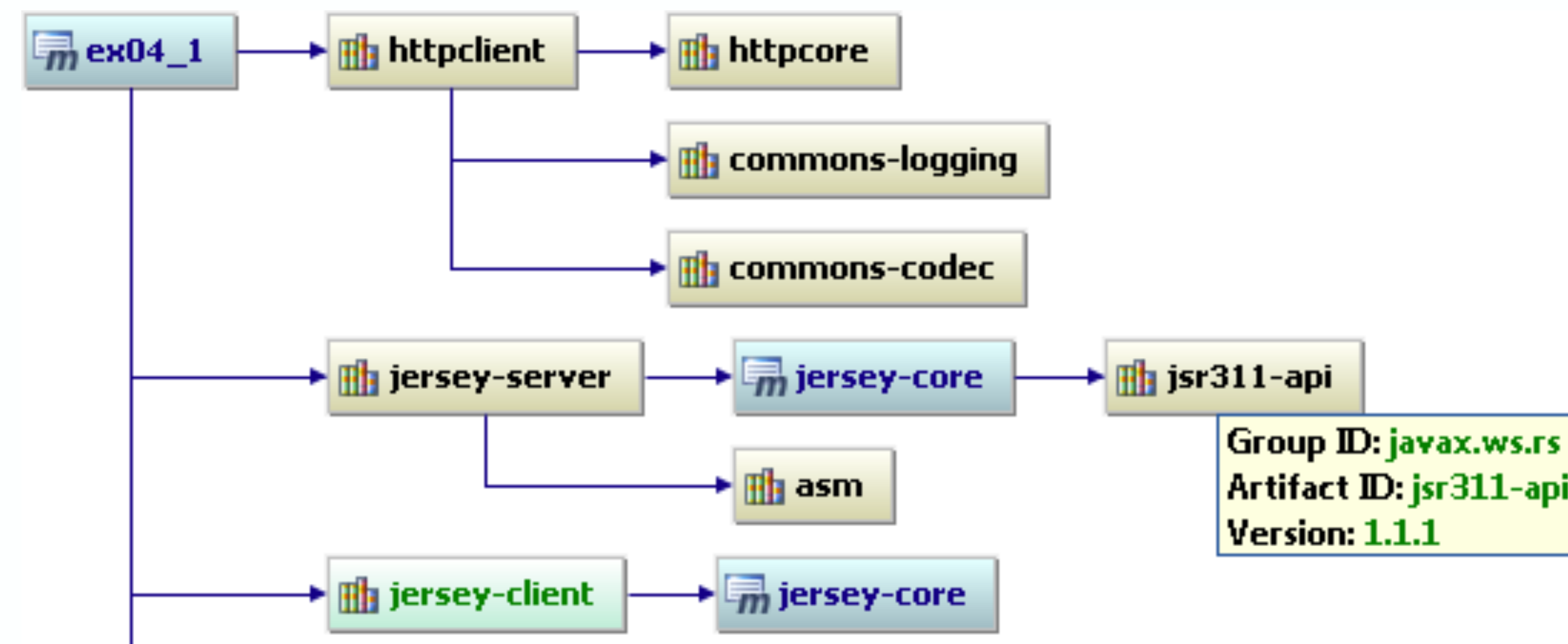


Gradle



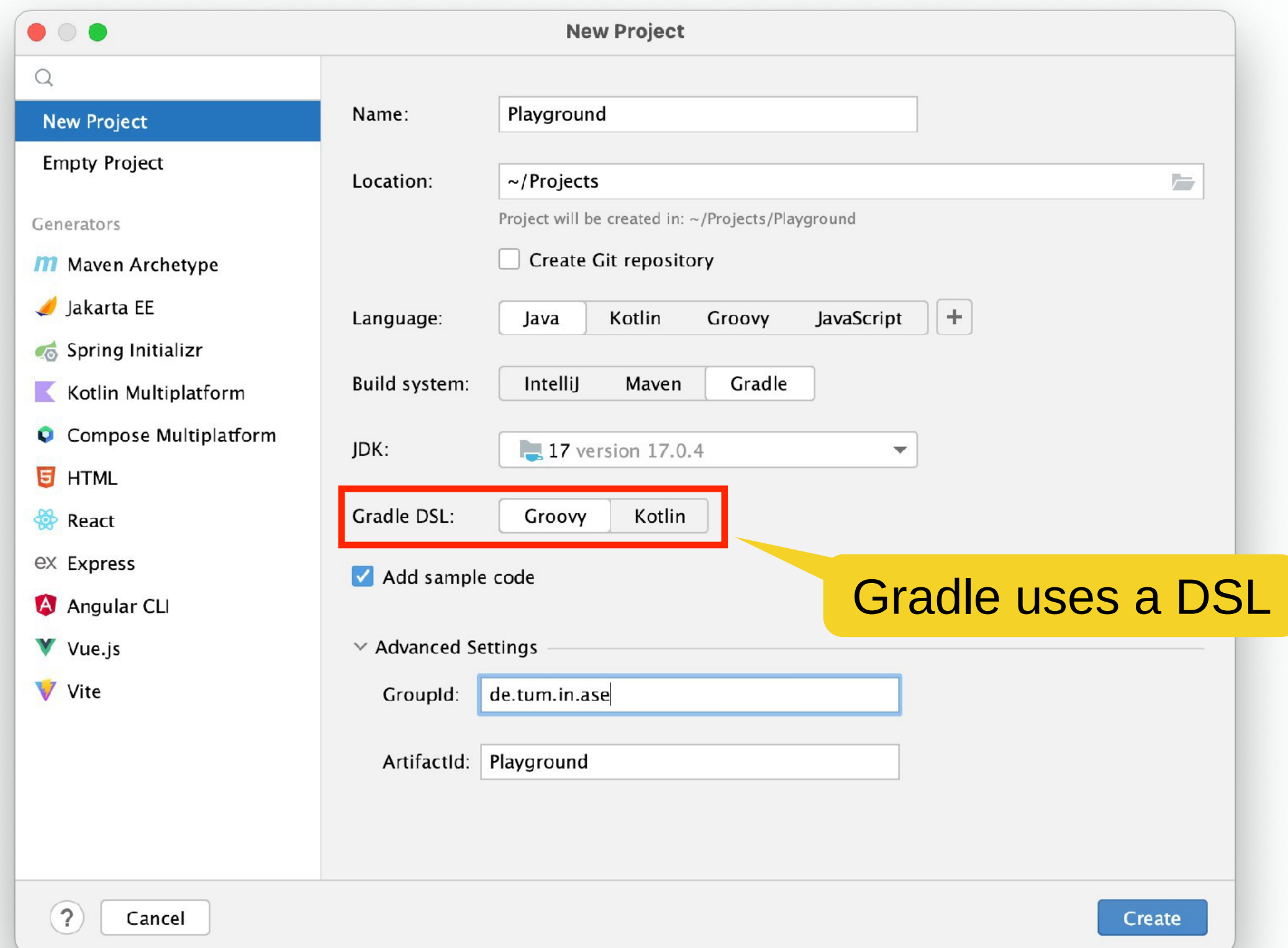
- Open-source **build automation tool** designed to be flexible enough to build almost any type of software
- **Resolve dependencies**: download **libraries, frameworks, and plugins** from a central repository → simplifies **reuse**
- **Build software** → typical tasks: clean, compile, build, test
- Documentation: https://docs.gradle.org/current/userguide/getting_started_eng.html

Dependency example



Gradle Java project

- Let's create a Gradle project together in IntelliJ



build.gradle

```
plugins {  
    id 'java'  
}  
  
group 'de.tum.in.ase'  
version '1.0-SNAPSHOT'  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    testImplementation 'org.junit.jupiter:junit-jupiter-api:5.8.1'  
    testRuntimeOnly 'org.junit.jupiter:junit-jupiter-engine:5.8.1'  
}  
  
test {  
    useJUnitPlatform()  
}
```

<https://github.com/ls1intum/java-json-demo>

Dependencies

- External libraries can be added easily
- Maven central repository search: <https://search.maven.org>
- Let's adapt **build.gradle** and add a common dependency: **json 20231013**

build.gradle

```
plugins {  
    id 'java'  
    id 'org.checkerframework' version '0.6.37'  
    id 'application'  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    implementation 'org.json:json:20231013'  
    checkerFramework 'org.checkerframework:checker:3.42.0'  
}  
  
application {  
    mainClass = 'de.tum.in.ase.Main'  
}
```

The **build.gradle** file contains some additional changes

Dependency to external framework

Movie class

```
import java.util.*;

public class Movie {

    private String imdbID;
    private String title;
    private String simplePlot;
    private String[] genres;
    private int year;
    private String rating;

    public Movie(String imdbID, String title, String simplePlot, String[] genres, int year, String rating) {
        this.imdbID = imdbID;
        this.title = title;
        this.simplePlot = simplePlot;
        this.genres = genres;
        this.year = year;
        this.rating = rating;
    }

    @Override
    public String toString() {
        return "Movie{" + "imdbID='" + imdbID + '\'' + ", title='" + title + '\''
            + ", simplePlot='" + simplePlot + '\'' + ", genres=" + Arrays.toString(genres)
            + ", year=" + year + ", rating='" + rating + '\'' + '}';
    }
}
```

Serialize: Movie → JSON

```
import org.json.*;
import java.util.*;

public class Movie {

    // ...

    //Serialize: Movie → JSON
    public JSONObject toJson() {
        JSONObject json = new JSONObject();
        json.put("imdbID", imdbID);
        json.put("title", title);
        json.put("genres", List.of(genres));
        json.put("simplePlot", simplePlot);
        json.put("year", year);
        json.put("rating", rating);
        return json;
    }
}
```

JSON can be seen as a mixture of maps (**JSONObject**) and arrays (**JSONArray**)

Deserialize: JSON → Movie

```
import org.json.*;
import java.util.*;

public class Movie {

    // ...

    // Deserialize: JSON → Movie
    public Movie(JSONObject jsonObject) {
        imdbID = jsonObject.getString("imdbID");
        title = jsonObject.getString("title");
        simplePlot = jsonObject.getString("simplePlot");
        JSONArray genreJsonArray = jsonObject.getJSONArray("genres");
        genres = new String[genreJsonArray.length()];
        for (int i = 0; i < genreJsonArray.length(); i++) {
            genres[i] = genreJsonArray.getString(i);
        }
        year = jsonObject.getInt("year");
        rating = jsonObject.getString("rating");
    }
}
```

JSON can be seen as a mixture of maps (**JSONObject**) and arrays (**JSONArray**)

Usage in Main

```
import org.json.JSONObject;

public class Main {

    public static void main(String[] args) {
        Movie hiddenFiguresMovie = new Movie("tt4846340", "Hidden Figures",
            "Three ...", new String[] { "History", "Drama" }, 2016, "7.8");

        String jsonString = serialize(hiddenFiguresMovie);
        System.out.println(jsonString);

        Movie anotherHiddenFigureMovie = deserialize(jsonString);
        System.out.println(anotherHiddenFigureMovie);
    }

    // Serialize: Movie → JSON String
    public static String serialize(Movie movie) {
        JSONObject jsonObject = movie.toJson();
        return jsonObject.toString(4); // pretty print
    }

    // Deserialize: JSON String → Movie
    public static Movie deserialize(String jsonString) {
        JSONObject jsonObject = new JSONObject(jsonString);
        return new Movie(jsonObject);
    }
}
```

Command

```
./gradlew run
```

Output

```
{
  "year": 2016,
  "imdbID": "tt4846340",
  "genres": [
    "History",
    "Drama"
  ],
  "rating": "7.8",
  "simplePlot": "Three ...",
  "title": "Hidden Figures"
}
Movie{imdbID='tt4846340', title='Hidden
Figures', simplePlot='Three ...',
genres=[History, Drama], year=2016,
rating='7.8'}
```